



AI-native Engineering Workflow Recommendation Worksheet*

Use this companion worksheet throughout the Codex in Developer Workflows course to build a practical AI-native engineering workflow recommendation for Cavendish Cables.

Your recommendation should explain where engineering work is losing time, which Codex collaboration pattern fits the workflow, what Codex can support, what engineers still own, what controls are needed, what evidence Codex should return, and what would justify expansion.

This is a working recommendation. It is not a technical implementation plan, security design, administration guide, or product configuration guide.

* Safe-use reminder: Use only fictional, sanitized, or approved training information when completing this worksheet. Do not include secrets, credentials, proprietary source code, confidential architecture details, customer data, sensitive deployment information, unapproved system or environment details, private customer identifiers, production credentials, or internal access tokens. All recommendations remain subject to human review, customer approval, and SME validation before real deployment.

1. Customer workflow friction

Describe where engineering work is losing time, continuity, or context.

Field	Response
Repeated workflow friction	
How the friction affects software delivery	
Why this friction is worth addressing	

Before moving on, check whether the friction is specific enough to support a workflow recommendation. For example, “reviewers lack implementation context” is more useful than “developers need to move faster.”

2. SDLC workflow map

Map the friction to the software-development lifecycle.

SDLC workflow moment	Where friction appears	What Codex could support	What engineers still own
Planning			
Implementation			
Review			
Testing			
Debugging			
Documentation			
Long-running work			
Context continuity or handoff			

Field	Response
Most important workflow moment to address first	
Why this workflow moment should be prioritized	

3. Collaboration-pattern selection

Select the collaboration pattern that fits the workflow.

Collaboration pattern	Best fit	Watch out for
Pair programming	The engineer needs interactive support to explore, debug, compare options, understand code, or stay close to the work.	Scope can become messy if the engineer does not provide enough context or a clear goal.
Governed delegation	Codex can work toward a bounded goal and return evidence for human review.	Requires clear goals, approved context, validation evidence, review ownership, and stop conditions.
Review checkpoint	Codex prepares evidence that supports a human decision.	Codex must not make the approval, merge, release, or risk decision.
Stronger manual workflow first	The workflow is high-risk, unclear, sensitive, production-impacting, or missing review and validation paths.	Do not force Codex into the workflow before it is clearer, safer, or better governed.

Recommended collaboration pattern

Select one or more

Pair programming	☐
Governed delegation	☐
Review checkpoint	☐
Stronger manual workflow first	☐

Field	Response
Why this pattern fits	
Why another pattern would be weaker	

4. Recommended pair-programming workflows

Identify workflows where Codex should support an engineer interactively, with the engineer steering the work in the moment.

Field	Response
Recommended pair-programming workflow	
Why pair programming fits	
What the engineer should actively steer	
What evidence or notes should be captured	

Pair programming may fit when the engineer needs to understand an unfamiliar function, explore implementation options, debug a local issue, draft or refine a small test, compare possible approaches, reason through a refactor, clarify a ticket before implementation, or inspect a small code path while staying close to the work.

5. Recommended governed-delegation workflows

Identify workflows where Codex can work toward a bounded goal and return evidence for human review.

Field	Response
Recommended governed-delegation workflow	
Why governed delegation fits	
What Codex can support	
What evidence Codex must return	

What the human reviewer decides	
---------------------------------	--

Governed delegation may fit when the task is clear, bounded, supported by approved context, verifiable, reviewable, owned by a human reviewer, and appropriate for Codex to investigate, draft, summarize, or prepare.

6. Workflows requiring stronger manual control

Identify workflows that should remain human-led, stay out of scope, or require stronger oversight before Codex is used.

Field	Response
Workflow that should remain human-led or out of scope for now	
Why stronger manual control is needed	
Readiness condition before reconsidering Codex use	

Stronger manual control may be needed when the workflow involves production approval, merge or release decisions, high-risk architecture decisions, sensitive systems or data, undefined business rules, broad uncontrolled scope, no review owner, no validation path, unclear approval boundaries, unclear escalation paths, production-impacting changes, or restricted repositories and environments

7. Codex support, engineer ownership, and controls

Use this section to keep accountability clear.

Recommended workflow	What Codex can support	What engineers still own	Controls needed
Pair-programming workflow			
Governed-delegation workflow			
Review-checkpoint workflow			
Stronger manual workflow			

Human-owned decision	Select if applicable	Notes
Prioritization	☐	
Architecture judgment	☐	
Code quality judgment	☐	
Review approval	☐	
Merge decision	☐	
Production release decision	☐	
Risk acceptance	☐	
Customer-impacting decision	☐	
Security-sensitive decision	☐	
Escalation decision	☐	
Accountability for outcomes	☐	
Other	☐	

Field	Response
Notes on human ownership	

8. Required context

Identify what Codex would need to know or inspect to support the recommended workflow.

Field	Response
Required context	
Context that should remain out of scope	
Context gaps to resolve	

Context quality check	Select when true
Relevant	☐
Approved	☐
Narrow enough	☐
Current	☐
Understandable	☐
Free of secrets or credentials	☐
Enough for Codex to work usefully	☐
Not broader than the workflow requires	☐

Required context may include relevant files, pull requests, tickets, logs, test output, error messages, review expectations, definition of done, engineering conventions, documentation standards, approved constraints or policies, architecture notes, or recent change context.

9. Recommended runtime or collaboration pattern

Select where the work should happen.

Runtime or collaboration pattern	Select one or more
Codex IDE extension	☐
Codex CLI	☐
ChatGPT desktop app with Codex	☐
Codex cloud or asynchronous workflow	☐
Native GitHub code review	☐
Review checkpoint	☐
Stronger manual workflow first	☐
Other	☐

Field	Response
Why this pattern fits the workflow	
Where Codex should work	
Where Codex should not work	
Checkpoint before continuing	

The runtime pattern should fit the workflow. It should not be selected only because a team prefers a tool or wants to try a new interface.

10. Task structure

Write the recommended Codex task as a clear delegation brief.

Use this structure when a workflow is suitable for governed delegation.

Delegation brief section	Prompt	Response
Goal	What should be true when the work is complete?	
Context	What files, tickets, logs, screenshots, docs, constraints, or patterns should Codex use?	
Scope: in scope	What is in scope?	
Scope: out of scope	What is out of scope?	
Validation	What tests, checks, diffs, summaries, or review evidence should Codex return?	
Stop conditions	When should Codex pause and ask for help?	

Before moving on, check whether the task is clear, bounded, context-supported, verifiable, and reviewable.

11. Validation evidence

Define what evidence Codex should return so a human can inspect the work.

Validation evidence	Select if required	Validation evidence	Select if required
Summary of work completed	☐	Files inspected	☐
Files changed	☐	Diff	☐
Test command	☐	Test result	☐
Command output	☐	Pull-request summary	☐
Review questions	☐	Missing validation evidence	☐
Assumptions	☐	Risks	☐
Open questions	☐	Recommended next step	☐
Handoff note	☐	Other	☐

Field	Response
Required validation evidence	
Evidence that would make the output not ready for review	
What the reviewer should inspect first	

A strong evidence requirement helps the reviewer understand what Codex inspected, what Codex produced, what was checked, what remains uncertain, and what requires human decision or escalation.

12. Review checkpoints

Define where human review should happen.

Review checkpoint decision	Response
Human review owner	
What the reviewer should inspect	
Decision the reviewer makes	
What remains human-owned	
Escalation path	

Escalation trigger	Select if applicable	Notes
Sensitive logic appears	☐	
Sensitive files or data appear	☐	
Scope is broader than expected	☐	
Validation fails	☐	
Evidence is missing	☐	

Ownership is unclear	☐	
Architecture impact is unclear	☐	
Production-impacting risk appears	☐	
Codex requests an action outside the approved boundary	☐	
Reviewer confidence is low	☐	
Other	☐	

13. Governance and operating boundaries

Define the boundaries needed to keep the workflow controlled.

Boundary decision	Response
Approved boundary	
Restricted actions	

Approval requirements	
Visibility or traceability needs	
Operating-boundary summary	

Consider which repository, team, branch, workflow, files, tools, or environments are in scope; which actions are restricted; what requires approval; what activity should be visible; and what should be traceable later.

14. Expansion evidence

Identify what evidence would support expanding Codex collaboration to another team, repository, or workflow.

Expansion evidence	Select if required	Notes
Outputs are useful to reviewers	☐	
Validation evidence is consistently returned	☐	

Human review ownership is clear	☐	
Restricted actions are respected	☐	
Teams understand when to pause or escalate	☐	
Workflow friction is reduced	☐	
Documentation or handoff quality improves	☐	
Governance expectations are followed	☐	
Review quality remains strong	☐	
Context continuity improves	☐	
The workflow remains clear, reviewable, validated, and governed	☐	

Other	☐	
-------	--------------------------	--

Field	Response
Evidence that would support expansion	
Evidence that would suggest pausing or refining before expansion	
Recommended next workflow, if expansion is justified	

Expansion should depend on repeated evidence, not enthusiasm alone.

15. Recommendation summary

Use this section to consolidate your decisions into a concise recommendation.

Recommendation element	Response
Workflow friction	
Pair-programming workflow	
Governed-delegation workflow	
Review-checkpoint workflow	
Workflow requiring stronger manual control	
Required context	
Runtime or collaboration pattern	
Validation evidence	

Review checkpoint	
Human-owned decisions	
Operating boundary	
Expansion evidence	

Suggested structure:

Rocco Labs recommends using Codex for [pair-programming workflow] and [governed-delegation workflow] because [workflow friction]. The first delegated workflow should stay within [boundary] and require [validation evidence] and [review checkpoint]. Engineers will continue to own [human-owned decisions]. Workflows involving [higher-risk area] should remain human-led until [readiness condition]. Expansion should depend on [evidence].

Field	Response
Recommendation summary	

16. Final readiness check

Before sharing your recommendation, check that each statement is true.

Readiness statement	Check
I can name the repeated engineering workflow friction.	☐
I can map the friction to planning, implementation, review, testing, debugging, documentation, long-running tasks, or context continuity.	☐
I can explain whether the work needs pair programming, governed delegation, review checkpoint, or stronger manual workflow.	☐
I can distinguish local review preparation from native GitHub code review and explain who retains approval and merge decisions.	☐
I can describe what Codex can support.	☐
I can explain what engineers still own.	☐
I can identify what controls are needed.	☐
I can describe what Codex needs to know.	☐
I can explain where Codex should work.	☐
I can identify what Codex should return for review.	☐
I can name the human review owner.	☐
I can explain what must remain human-owned.	☐
I can identify what should trigger escalation.	☐
I can describe what evidence would justify expansion.	☐
I have avoided positioning Codex as uncontrolled automation.	☐
I have avoided unsupported productivity, replacement, or autonomous engineering claims.	☐

I have not included real customer data, credentials, secrets, proprietary code, or sensitive technical details.

☐

If any answer is unclear, the recommendation needs more discovery, refinement, or SME validation before it is ready to share.

OpenAI

